

SIITHSYSTEMS Desktop Commander Fork

Remote MCP Usage, Pairing, Security,
Deployment, and Self-Hosting Architecture
Guide

SIITHSYSTEMS

21 September 2026

- Document status
- 1. Executive summary
 - 1.1 Local MCP mode
 - 1.2 Remote MCP mode
- 2. What was forked
- 3. What is open source and what is not
 - 3.1 Included in DesktopCommanderMCP
 - 3.2 Not included
- 4. Official user experience
- 5. Official device setup
 - 5.1 Requirements
 - 5.2 Start the device
 - 5.3 What the user sees
- 6. OAuth Device Authorization + PKCE
- 7. Device authorization API contract
 - 7.1 Start pairing
 - 7.2 Poll for approval
- 8. Why the flow works on servers and headless systems
- 9. Remote MCP bootstrap
- 10. Local credential persistence
- 11. Remote CLI options
- 12. Token rotation
- 13. How the local MCP engine is launched
- 14. Source-fork development
- 15. Custom compatible backend
- 16. ChatGPT connector flow
- 17. Realtime architecture visible in the device source
- 18. Device registry fields
- 19. Reachability
- 20. Remote call lifecycle
- 21. Exactly-once safeguards
- 22. End-to-end call sequence
- 23. Multi-device model
- 24. Secret inventory
 - 24.1 Client-side sensitive values verified in the source
 - 24.2 Public/non-secret values
- 25. Server-side secrets for an independent implementation
- 26. Secret storage
- 27. Recommended Vault paths
- 28. Can this fork be installed on a server?
 - 28.1 Server as the controlled device
 - 28.2 Server as a replacement cloud relay
- 29. Minimum self-hosted architecture
- 30. Suggested URLs
- 31. OAuth requirements
- 32. Device-code security
- 33. Supabase-compatible implementation
- 34. Conceptual mcp_devices schema
- 35. Conceptual mcp_remote_calls schema
- 36. Row Level Security
- 37. RHEL 10.x setup
- 38. Production reverse proxy example
- 39. Container deployment principles
- 40. Security trust model
- 41. Dedicated Linux user
- 42. Network security checklist
- 43. Audit logging
- 44. Local audit data
- 45. Service management
- 46. Revocation
- 47. Safe diagnostics
- 48. Pairing troubleshooting
- 49. Device offline troubleshooting
- 50. Local MCP unavailable
- 51. Custom backend troubleshooting order

- 52. Production test matrix
 - Authentication
 - Authorization
 - Transport
 - Tool execution
 - Security
- 53. CI/CD recommendations
- 54. Keeping the fork synchronized
- 55. GitHub/GitLab policy
- 56. License boundary
- 57. Recommended SIITHSYSTEMS implementation phases
 - Phase A - Preserve fork
 - Phase B - Compatibility harness
 - Phase C - Identity and pairing
 - Phase D - Device registry/realtime
 - Phase E - Remote MCP gateway
 - Phase F - ChatGPT qualification
 - Phase G - Hardening
- 58. Proposed SIITHSYSTEMS environment contract
- 59. Definition of green
- 60. Quick reference
- 61. Maintainer source map
- 62. Secret-scan reminder
- 63. Final engineering conclusion
- 64. Change-control note

Document status

Purpose: Detailed operating, security, deployment, and engineering guide for the SIITHSYSTEMS fork of DesktopCommanderMCP.

GitHub fork: <https://github.com/siithsystems/DesktopCommanderMCP>

GitLab engineering copy: <https://gitlab.com/siithsystems/DesktopCommanderMCP>

Upstream: <https://github.com/wonderwhy-er/DesktopCommanderMCP>

Verified upstream baseline before this documentation:

c4c3d6b0c6e6d3f69291ec6aa7c439e680708a70

Security rule: Never commit live passwords, private keys, API tokens, OAuth access/refresh tokens, database passwords, service-role keys, cookie secrets, signing keys, or other production secrets. This guide documents secret names, purpose, lifecycle, storage, and rotation using placeholders only.

Critical scope boundary: The MIT-licensed DesktopCommanderMCP repository contains the local MCP tool engine and the remote-device agent. It does **not** contain the proprietary hosted Remote Desktop Commander control plane/relay behind `mcp.desktopcommander.app` and `auth.desktopcommander.app`. The public `desktop-commander/remote-desktop-commander` repository contains public manifests/documentation, not the hosted service source. Therefore this fork can run the local engine and remote device and can connect to the official hosted service, but a fully independent SIITHSYSTEMS service requires a separately implemented compatible Remote MCP/OAuth/device-pairing/relay backend.

1. Executive summary

Desktop Commander has two operating models.

1.1 Local MCP mode

The AI client and Desktop Commander run on the same computer. The MCP client launches the Desktop Commander executable over stdio and directly receives filesystem, terminal, process, search, structured-document, and configuration tools.

```
AI MCP client
|
| stdio MCP
v
Desktop Commander local MCP
|
+-- filesystem
+-- terminal/processes
+-- search
+-- PDF/DOCX/XLSX
+-- configuration/history
```

No cloud relay is required.

1.2 Remote MCP mode

The AI application connects to a hosted Remote MCP endpoint over HTTPS. A lightweight device process runs on the computer being controlled. The server relays authenticated tool calls to that device; the device forwards them into a local Desktop Commander MCP child process and sends results back.

```
ChatGPT / AI client
|
| HTTPS / Streamable HTTP MCP
| OAuth 2.0
v
Remote MCP gateway/control plane
|
| authenticated realtime transport
| + durable call state
v
Remote device agent
|
| stdio MCP
v
Desktop Commander local MCP
|
+-- local files
+-- terminal
+-- processes
+-- search
+-- documents
```

The target machine performs the actual operations. The cloud layer is the identity, routing, state, and relay plane.

2. What was forked

The SIITHSYSTEMS GitHub repository is a genuine GitHub fork of wonderwhy-er/DesktopCommanderMCP.

The fork contains the MIT-licensed local Desktop Commander server and the remote-device client under:

```
src/remote-device/  
src/npm-scripts/remote.ts
```

The packaged remote device is started with:

```
npx @wonderwhy-er/desktop-commander@latest remote
```

The project currently requires Node.js 18 or newer.

3. What is open source and what is not

3.1 Included in DesktopCommanderMCP

The fork includes:

- Local MCP server.
- Filesystem tools.
- Terminal/process tools.
- Search.
- Process/session management.
- PDF, Excel, and DOCX capabilities provided by the project.
- Configuration management.
- Local tool history/audit features.
- Remote-device CLI.
- OAuth device-flow client logic.
- PKCE generation.
- Device credential persistence.
- Supabase client integration for the remote-device transport.
- Device presence and heartbeat handling.
- Reconnection and refresh-token handling.
- Remote call claiming and result updates.
- Stdio bridge to the local Desktop Commander MCP server.

3.2 Not included

The open-source fork does not contain the official production implementation of:

- The `mcp.desktopcommander.app` dashboard.
- `auth.desktopcommander.app`.
- The hosted Streamable HTTP MCP server.
- Server-side OAuth/control-plane code.
- Server-side device dispatcher.
- Production database migrations and policies.
- Production billing/account administration.
- Abuse prevention and operational systems.

The public remote-desktop-commander repository states that the hosted service implementation is proprietary.

Engineering consequence: do not deploy this fork to a server and assume that server automatically becomes a replacement for the official Remote Desktop Commander SaaS. The fork is the local/device side. A compatible remote server must be built separately.

4. Official user experience

The official remote flow matches the behavior observed in the ChatGPT plugin/connector experience:

1. The AI-side connector is selected.
2. The user is sent to an authorization page.
3. The target computer runs a command that starts the device agent.
4. The agent receives a short pairing code.
5. A browser page opens automatically where possible.
6. The browser and terminal display the same code.
7. The user signs in and verifies the code.
8. The local device polls until authorization is complete.
9. The device registers and becomes online.
10. The AI-side OAuth authorization finishes.
11. The browser returns to the AI application.
12. Remote MCP tools become usable against the paired machine.

This is not traditional screen sharing. It is authenticated remote MCP tool execution.

5. Official device setup

5.1 Requirements

- Node.js 18+.
- Internet access.
- An account on the hosted Remote Desktop Commander service.
- A supported AI client for the Remote MCP connector.

Check Node:

```
node --version  
npm --version
```

5.2 Start the device

```
npx @wonderwhy-er/desktop-commander@latest remote
```

The terminal should enter the device authorization flow.

5.3 What the user sees

The device prints instructions similar to:

```
Starting device authorization flow...  
Requesting device code...  
Device code received  
  
Please complete authentication:  
1. Verify this device in your browser:  
   https://.../verify-device?user_code=ABCD-EFGH  
2. Make sure the code matches:  
   ABCD-EFGH  
  
Waiting for authorization...
```

The user signs in and approves only when the browser code exactly matches the terminal code.

Once authorization succeeds, the device establishes the remote channel, registers its capabilities, and ultimately reports that the device is ready.

6. OAuth Device Authorization + PKCE

The open-source device client implements OAuth-style device authorization with PKCE.

Verified public client values:

```
client_id = mcp-device  
scope = mcp:tools  
code_challenge_method = S256
```

The PKCE verifier is generated locally using cryptographically random bytes. A SHA-256 challenge is sent to the server.

The verifier must remain local until the poll/token exchange.

7. Device authorization API contract

7.1 Start pairing

```
POST <MCP_SERVER_URL>/device/start
Content-Type: application/json
```

Observed request shape:

```
{
  "client_id": "mcp-device",
  "scope": "mcp:tools",
  "device_name": "<HOSTNAME>",
  "device_type": "mcp",
  "device_id": "<OPTIONAL_EXISTING_DEVICE_ID>",
  "code_challenge": "<PKCE_S256_CHALLENGE>",
  "code_challenge_method": "S256"
}
```

Expected response concept:

```
{
  "device_code": "<OPAQUE_DEVICE_CODE>",
  "user_code": "ABCD-EFGH",
  "verification_uri": "https://mcp.example.com/verify-device",
  "verification_uri_complete": "https://mcp.example.com/verify-device?user_code=ABCD-EFGH",
  "expires_in": 900,
  "interval": 5
}
```

7.2 Poll for approval

```
POST <MCP_SERVER_URL>/device/poll
Content-Type: application/json
```

Request:

```
{
  "device_code": "<OPAQUE_DEVICE_CODE>",
  "client_id": "mcp-device",
  "code_verifier": "<PKCE_VERIFIER>"
}
```

Pending states include:

```
{"error": "authorization_pending"}
```

and:

```
{"error": "slow_down"}
```

Successful response contains an access token, refresh token, and may contain the device ID.

8. Why the flow works on servers and headless systems

The device does not require a localhost OAuth callback.

It prints a verification URL and polls the server. Therefore the target can be:

- A workstation.
- A remote Linux host.
- A headless server.
- A VM.
- A container environment.

If the server cannot open a browser, copy the verification URL to a trusted workstation/browser, authenticate there, verify the matching code, and return to the terminal.

9. Remote MCP bootstrap

The current device reads its remote service base from:

```
MCP_SERVER_URL
```

If unset, the default is:

```
https://mcp.desktopcommander.app
```

Before authentication it performs:

```
GET <MCP_SERVER_URL>/api/mcp-info
```

The source expects values equivalent to:

```
{
  "supabaseUrl": "https://<PROJECT>.supabase.co",
  "supabasePublishableKey": "<PUBLIC_PUBLISHABLE_KEY>"
}
```

This endpoint is intentionally public in the client design.

A Supabase publishable/anon key is not a service-role secret. Its safety depends on correct Row Level Security.

10. Local credential persistence

Default credential file:

```
~/.desktop-commander-device/device.json
```

The source writes it with Unix mode 0600.

Conceptual structure:

```
{
  "deviceId": "<DEVICE_ID>",
  "session": {
    "access_token": "<REDACTED>",
    "refresh_token": "<REDACTED>"
  }
}
```

Never commit this file.

Check permissions without printing contents:

```
stat -c '%a %U %G %n' ~/.desktop-commander-device/device.json
```

Expected permission:

```
600
```

11. Remote CLI options

Help:

```
npx @wonderwhy-er/desktop-commander@latest remote --help
```

Debug:

```
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

Disable persisted session:

```
npx @wonderwhy-er/desktop-commander@latest remote --no-persist-session
```

Disable the macOS no-sleep helper:

```
npx @wonderwhy-er/desktop-commander@latest remote --disable-no-sleep
```

Remove locally persisted remote credentials:

```
npx @wonderwhy-er/desktop-commander@latest remote --logout
```

Local logout does not substitute for server-side device revocation.

12. Token rotation

The remote client deliberately handles refresh-token rotation.

The source currently:

- Disables the default Supabase auto-refresh timer.
- Runs its own refresh cadence.
- Uses an approximately 45-minute refresh interval.
- Reauthorizes the realtime connection with the current access token.
- Persists rotated refresh tokens.
- Tries one bounded recovery after an unexpected sign-out.
- Goes offline if the session cannot be recovered.

A compatible backend must treat refresh tokens as high-value, rotating credentials.

13. How the local MCP engine is launched

The remote-device process is a bridge, not the complete tool engine.

It resolves a local Desktop Commander MCP executable in this order:

1. A development build under the checked-out project.
2. A globally installed desktop-commander command.

It then launches the local MCP child over stdio using the Model Context Protocol SDK.

The child receives:

```
DC_REMOTE_DEVICE=true
```

This identifies the remote-device context and suppresses local-only behavior that would not make sense for the remote user.

14. Source-fork development

Clone SIITHSYSTEMS:

```
git clone https://github.com/siithsystems/DesktopCommanderMCP.git
cd DesktopCommanderMCP
```

Install:

```
npm ci
```

Build:

```
npm run build
```

Run local MCP:

```
npm start
```

Run device during development:

```
npm run device:start
```

Create a global development link:

```
npm run link:local
```

Then:

```
desktop-commander remote
```

15. Custom compatible backend

The remote-device source makes the server replaceable through MCP_SERVER_URL.

Example:

```
MCP_SERVER_URL=https://mcp.siithsystems.com \  
desktop-commander remote --debug
```

or:

```
MCP_SERVER_URL=https://mcp.siithsystems.com \  
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

Do not point production devices at a custom endpoint until it passes compatibility and security tests.

16. ChatGPT connector flow

Public Remote Desktop Commander documentation describes the ChatGPT connection approximately as:

1. Open ChatGPT Settings.
2. Open Apps & Connectors.
3. Open Advanced settings.
4. Enable Developer mode where the plan/account provides it.
5. Create a connector.
6. Enter: `~text-https://mcp.desktopcommander.app/mcp~`
7. Complete OAuth authorization.
8. Return to ChatGPT.
9. Use tools exposed by the connector.

A future SIITHSYSTEMS server would use a URL such as:

```
https://mcp.siithsystems.com/mcp
```

only after it implements the remote MCP and OAuth contracts expected by ChatGPT.

17. Realtime architecture visible in the device source

The remote device uses Supabase Auth, PostgREST/database operations, and Supabase Realtime.

Observed tables:

```
mcp_devices  
mcp_remote_calls
```

Observed private channel:

```
user:<USER_ID>
```

Observed Broadcast event:

```
new_call
```

Presence uses the device ID as its key.

18. Device registry fields

Observed client-side device concepts include:

```
id
user_id
device_name
capabilities
status
last_seen
```

Statuses include:

```
online
offline
```

Capability data contains the app version and, after end-to-end transport proof:

```
transport_broadcast_v1 = true
```

The client does not advertise the broadcast capability merely because it has started. It waits until the private channel and Presence are genuinely working.

19. Reachability

The client combines:

- Realtime channel state.
- Presence.
- Realtime heartbeat evidence.
- Database last_seen.
- Device status.
- Reconnect attempts.
- Capability withdrawal/recovery.

It detects unhealthy and half-open sockets and recreates channels with bounded/jittered backoff.

A self-hosted backend should use Presence/realtime reachability as a primary signal and last_seen as a durable fallback.

20. Remote call lifecycle

A compatible server should persist a call before dispatch.

The open-source device expects database-backed remote-call state. Observed concepts include:

```
pending
executing
failed
```

and terminal result fields such as:

```
status
result
error_message
completed_at
```

Recommended complete lifecycle:

```
pending
|
v
executing
|
+--> completed
+--> failed
+--> cancelled
+--> timed_out
```

21. Exactly-once safeguards

The device conditionally claims a call from pending to executing.

Conceptual SQL:

```
update mcp_remote_calls
set status = 'executing',
    started_at = now()
where id = :call_id
    and device_id = :device_id
    and status = 'pending'
returning *;
```

If no row is returned, the tool must not execute.

This prevents duplicate Broadcast delivery from executing side effects twice.

The device also keeps a bounded in-memory set of recent call IDs.

22. End-to-end call sequence

1. ChatGPT sends a tool call to the Remote MCP endpoint.
2. Remote MCP authenticates the AI-side user.
3. The server selects the target device.
4. Server creates `mcp_remote_calls` row as pending.
5. Server broadcasts `new_call` with `call_id` and `device_id`.
6. Device receives the doorbell.
7. Device atomically claims pending -> executing.
8. Device forwards tool + arguments to local Desktop Commander over `stdio MCP`.
9. Local Desktop Commander executes on the computer.
10. Local MCP returns result/error.
11. Device writes terminal call state/result.
12. Remote MCP completes the waiting MCP request.
13. ChatGPT receives the tool result.

23. Multi-device model

Recommended device record:

```
user_id
device_id
device_name
platform
app_version
status
last_seen
capabilities
revoked_at
```

Recommended dashboard functions:

- List devices.
- Rename.
- Choose default.
- Route to named device.
- Show online/offline.
- Show last seen.
- Revoke one.
- Revoke all.
- Show agent version.
- Show transport health.

Use stable device IDs internally, never only human-readable names.

24. Secret inventory

24.1 Client-side sensitive values verified in the source

Item	Sensitive	Lifecycle	Storage
access_token	Yes	Short-lived	Memory and optionally device.json
refresh_token	Highly sensitive	Longer-lived, rotating	Memory and optionally device.json
PKCE code_verifier	Yes, ephemeral	One device flow	Memory only
device_code	Temporary credential	Minutes	Memory
user_code	Temporary verification code	Minutes	Terminal/browser
device ID	Identifier	Persistent	device.json

24.2 Public/non-secret values

Item	Secret	Notes
MCP_SERVER_URL	No	Public service base URL
Supabase URL	No	Public endpoint
Supabase publishable/anon key	No by design	Must be protected by RLS
client_id=mcp-device	No	Public client identifier
scope=mcp:tools	No	Requested scope

25. Server-side secrets for an independent implementation

The following are **recommended architecture secret names**, not values extracted from the proprietary service:

```
DATABASE_URL
DATABASE_PASSWORD
SUPABASE_SERVICE_ROLE_KEY
OAUTH_JWT_SIGNING_PRIVATE_KEY
OAUTH_JWT_SIGNING_KEY_ID
SESSION_ENCRYPTION_KEY
DEVICE_CODE_HMAC_KEY
REFRESH_TOKEN_PEPPER
CSRF_SECRET
COOKIE_SECRET
MCP_INTERNAL_SIGNING_KEY
GOOGLE_OAUTH_CLIENT_SECRET
SMTP_PASSWORD
TLS_PRIVATE_KEY
```

Never put actual values in Git, Markdown, PDF, issues, screenshots, or CI logs.

26. Secret storage

Development:

- ignored .env.local;
- local secret manager;
- never commit .env.

Production:

- HashiCorp Vault or another dedicated secret manager;
- protected CI/CD secret references;
- runtime injection;
- narrow per-service permissions;
- scheduled and incident-driven rotation.

Example only:

```
PUBLIC_BASE_URL=https://mcp.siithsystems.com
SUPABASE_URL=https://example.supabase.co
SUPABASE_PUBLISHABLE_KEY=<PUBLIC>

DATABASE_URL=__FROM_VAULT__
SUPABASE_SERVICE_ROLE_KEY=__FROM_VAULT__
OAUTH_JWT_SIGNING_PRIVATE_KEY=__FROM_VAULT__
SESSION_ENCRYPTION_KEY=__FROM_VAULT__
DEVICE_CODE_HMAC_KEY=__FROM_VAULT__
GOOGLE_OAUTH_CLIENT_SECRET=__FROM_VAULT__
SMTP_PASSWORD=__FROM_VAULT__
```

27. Recommended Vault paths

```
secret/siithsystems/desktop-commander/prod/database  
secret/siithsystems/desktop-commander/prod/oauth  
secret/siithsystems/desktop-commander/prod/supabase  
secret/siithsystems/desktop-commander/prod/google  
secret/siithsystems/desktop-commander/prod/smtp
```

The remote device must never receive database-admin credentials, service-role keys, OAuth signing private keys, SMTP passwords, or server encryption keys.

28. Can this fork be installed on a server?

Yes, in two different senses.

28.1 Server as the controlled device

If a Linux server is the computer ChatGPT should control:

```
npx @wonderwhy-er/desktop-commander@latest remote
```

Pair it through the browser/device code.

That uses the official hosted relay.

28.2 Server as a replacement cloud relay

No, not from this repository alone.

The official cloud relay source is not included. SIITHSYSTEMS must build a compatible server-side control plane.

29. Minimum self-hosted architecture

```
ChatGPT / AI clients
|
| Streamable HTTP MCP + OAuth
v
SIITHSYSTEMS Remote MCP Gateway
|
+-- /mcp
+-- OAuth authorize/token/revoke
+-- /device/start
+-- /device/poll
+-- /api/mcp-info
+-- device routing
+-- call lifecycle
+-- timeout/cancellation
+-- rate limiting/audit
|
+-----+
|       |
v       v
Identity/Auth   Realtime/DB
                 - Presence
                 - Broadcast
                 - mcp_devices
                 - mcp_remote_calls
                 |
                 v
                 Remote device
                 |
                 | stdio MCP
                 v
                 Desktop Commander
```

30. Suggested URLs

```
https://mcp.siithsystems.com
https://auth.siithsystems.com      # optional
https://remote.siithsystems.com    # optional dashboard
```

Compatibility routes required by the current device:

```
GET  /api/mcp-info
POST /device/start
POST /device/poll
```

Remote MCP route:

```
/mcp
```

Recommended management routes:

```
GET    /api/devices
PATCH /api/devices/:id
DELETE /api/devices/:id
POST   /api/devices/revoke-all
GET    /api/account
POST   /oauth/authorize
POST   /oauth/token
POST   /oauth/revoke
```

31. OAuth requirements

The remote MCP service should provide:

- HTTPS.
- Authorization endpoint.
- Token endpoint.
- PKCE.
- State/CSRF protection.
- Exact redirect-URI allowlisting.
- Scope validation.
- Expiring access tokens.
- Refresh-token rotation.
- Revocation.
- Per-user authorization.
- Separation between AI-client OAuth sessions and device-agent sessions.

Do not use one universal bearer token for all customers/devices.

32. Device-code security

Generate:

- Cryptographically random opaque device_code.
- Short human-readable user_code.
- Short expiry.
- Enforced polling interval.
- PKCE binding.
- One-time approval.
- Brute-force protection.
- Rate limits.
- Audit events.
- Account binding.

Verification UI should show device name, platform where appropriate, code, request time, account, Approve and Cancel.

33. Supabase-compatible implementation

The shortest route to compatibility with the existing device is to provide:

- Supabase Auth-compatible user sessions.
- PostgREST-compatible database access.
- Supabase Realtime Broadcast/Presence.
- Correct Row Level Security.

A custom bootstrap response can expose only the public Supabase URL and publishable key.

Do not expose service-role credentials.

34. Conceptual mcp_devices schema

This is a recommended clean-room schema inferred from client behavior, not the proprietary service schema.

```
create table mcp_devices (  
  id uuid primary key,  
  user_id uuid not null,  
  device_name text not null,  
  capabilities jsonb not null default '{}':::jsonb,  
  status text not null default 'offline',  
  last_seen timestampz not null default now(),  
  created_at timestampz not null default now(),  
  updated_at timestampz not null default now(),  
  revoked_at timestampz null  
);
```

35. Conceptual mcp_remote_calls schema

```
create table mcp_remote_calls (  
  id uuid primary key,  
  user_id uuid not null,  
  device_id uuid not null references mcp_devices(id),  
  tool_name text not null,  
  arguments jsonb not null default '{}'::jsonb,  
  status text not null default 'pending',  
  result jsonb null,  
  error_message text null,  
  created_at timestampz not null default now(),  
  started_at timestampz null,  
  completed_at timestampz null  
);
```

Add indexes, constraints, retention policies, and RLS.

36. Row Level Security

Minimum policies:

- User A cannot read User B's devices.
- User A cannot mutate User B's calls.
- Device A cannot claim Device B's calls.
- Revoked device cannot restore itself.
- Publishable key without a valid user session reveals no private rows.
- Result writes must be scoped by user + device + call.
- Service-role key is server-only.

Cross-account negative tests are mandatory.

37. RHEL 10.x setup

Check runtime:

```
node --version  
npm --version
```

Run interactively first:

```
npx @wonderwhy-er/desktop-commander@latest remote
```

After successful initial authorization, a user-level systemd unit can keep it running.

Example:

```
[Unit]  
Description=SIITHSYSTEMS Desktop Commander Remote Device  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple  
ExecStart=/usr/bin/env npx -y @wonderwhy-er/desktop-commander@latest remote  
Restart=on-failure  
RestartSec=5  
Environment=NODE_ENV=production  
  
[Install]  
WantedBy=default.target
```

Commands:

```
systemctl --user daemon-reload  
systemctl --user enable --now siithsystems-desktop-commander-remote.service  
systemctl --user status siithsystems-desktop-commander-remote.service
```

For a custom backend add:

```
Environment=MCP_SERVER_URL=https://mcp.siithsystems.com
```

only after the backend is compatible.

38. Production reverse proxy example

```
server {
    listen 443 ssl http2;
    server_name mcp.siithsystems.com;

    ssl_certificate      /etc/letsencrypt/live/mcp.siithsystems.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/mcp.siithsystems.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:18090;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}
```

Validate proxy buffering and long-running Streamable HTTP behavior with the actual MCP implementation.

39. Container deployment principles

Recommended topology:

```
reverse proxy
|
remote MCP gateway
|
+-- PostgreSQL
+-- Redis (optional)
+-- Supabase-compatible services or equivalent
+-- OAuth/identity
```

Expose only 443 publicly unless another port is explicitly required.

Keep PostgreSQL, Redis, admin ports, and internal realtime infrastructure private.

Use non-root containers, health checks, resource limits, immutable images, and runtime secret injection.

40. Security trust model

Remote Desktop Commander is high privilege because tools execute with the operating-system permissions of the target user.

Therefore:

- Treat the connected AI account as a privileged control credential.
- Protect the Remote MCP account with MFA.
- Pair only computers you control.
- Stop the device when it is not needed.
- Use a VM/container for high-risk work.
- Prefer a dedicated unprivileged OS account.
- Treat command blocklists as guardrails, not a sandbox.
- Avoid running the agent as root.
- Keep sensitive paths outside the permitted work area where possible.

41. Dedicated Linux user

Example:

```
sudo useradd --create-home --shell /bin/bash dcagent
sudo mkdir -p /srv/ai-workspace
sudo chown dcagent:dcagent /srv/ai-workspace
```

Do not casually add the agent user to wheel, docker, libvirt, or equivalent high-privilege groups.

If privileged operations are necessary, use narrow sudoers entries.

42. Network security checklist

- TLS 1.2 minimum; prefer TLS 1.3.
- HSTS after validation.
- Secure cookies.
- Appropriate SameSite policy.
- OAuth state + PKCE.
- Strict redirect URI allowlist.
- Login/device-code/token rate limits.
- User-code brute-force protection.
- Refresh-token replay detection.
- Signing-key rotation with key IDs.
- Pair/revoke audit events.
- Authorization-header redaction.
- No bearer or refresh tokens in logs.
- Separate operational metadata from tool payload data.

43. Audit logging

Recommended hosted audit metadata:

```
timestamp
user_id
device_id
tool_name
status
duration_ms
request_id
transport
client application
error class
```

Avoid retaining raw:

```
file contents
terminal output
authorization headers
refresh tokens
passwords
private keys
secret-bearing tool arguments
```

If payload logging is needed for debugging, make it opt-in, encrypted, access-controlled, and short-lived.

44. Local audit data

Desktop Commander maintains local tool-call history.

Treat the local configuration/log directory as sensitive because tool arguments may themselves contain confidential values.

Do not automatically ship full local command history to a centralized log service.

45. Service management

Start:

```
systemctl --user start siithsystems-desktop-commander-remote.service
```

Stop:

```
systemctl --user stop siithsystems-desktop-commander-remote.service
```

Status:

```
systemctl --user status siithsystems-desktop-commander-remote.service
```

Logs:

```
journalctl --user -u siithsystems-desktop-commander-remote.service -f
```

Stopping the agent removes the remote execution path from that machine.

46. Revocation

Use both local and server-side controls.

Local credential removal:

```
npx @wonderwhy-er/desktop-commander@latest remote --logout
```

Server side should support:

- Revoke one device.
- Revoke all devices.
- Revoke AI connector authorization.
- Revoke refresh-token family.
- Force sign-out after suspected compromise.

47. Safe diagnostics

Process check:

```
pgrep -af 'desktop-commander.*remote'
```

Credential metadata without contents:

```
test -f ~/.desktop-commander-device/device.json \
&& stat -c '%a %U %G %n' ~/.desktop-commander-device/device.json
```

DNS:

```
getent ahosts mcp.siithsystems.com
```

TLS:

```
curl -I https://mcp.siithsystems.com/
```

Avoid printing device.json to shared terminals/logs because it can contain active tokens.

48. Pairing troubleshooting

If the browser does not open:

1. Copy the verification URL.
2. Open it manually on a trusted browser.
3. Sign in.
4. Verify the short code.
5. Approve.
6. Return to the terminal.

If codes do not match:

- Do not approve.
- Stop the process with Ctrl+C.
- Start pairing again.

49. Device offline troubleshooting

Check the process:

```
pgrep -af 'desktop-commander.*remote'
```

Debug:

```
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

With systemd:

```
systemctl --user restart siithsystems-desktop-commander-remote.service  
journalctl --user -u siithsystems-desktop-commander-remote.service -n 200 --no-pager
```

A server-side revoked device should require a fresh authorization rather than silently recreating itself.

50. Local MCP unavailable

Install globally:

```
npm install -g @wonderwhy-er/desktop-commander
command -v desktop-commander
```

Or use the SIITHSYSTEMS fork:

```
git clone https://github.com/siithsystems/DesktopCommanderMCP.git
cd DesktopCommanderMCP
npm ci
npm run build
npm link
```

Then restart the remote device.

51. Custom backend troubleshooting order

Run:

```
MCP_SERVER_URL=https://mcp.siithsystems.com \  
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

Validate in order:

1. GET /api/mcp-info.
2. Public JSON field names.
3. POST /device/start.
4. PKCE challenge storage.
5. Verification UI.
6. Device approval.
7. POST /device/poll.
8. Access/refresh session.
9. Authenticated database access.
10. Private realtime channel.
11. Presence.
12. mcp_devices RLS.
13. mcp_remote_calls RLS.
14. new_call Broadcast.
15. Call claim.
16. Local MCP execution.
17. Result update.

52. Production test matrix

Authentication

- Successful pairing.
- Expired device code.
- Wrong code.
- Wrong PKCE verifier.
- Reused code.
- Revoked device.
- Refresh-token rotation.
- Refresh-token replay.
- OAuth redirect-URI validation.
- OAuth state/PKCE.

Authorization

- User A cannot see User B devices.
- User A cannot update User B calls.
- Device A cannot claim Device B calls.
- Revoked device cannot return online.
- Publishable key alone reveals no private data.

Transport

- Realtime disconnect/reconnect.
- Half-open socket.
- Duplicate Broadcast.
- Missing Broadcast.
- Presence failure.
- Server restart.
- Device restart.
- Network partition.
- Significant clock skew.

Tool execution

- File read.
- File write.
- Search.
- Long-running process.
- Interactive process.
- PDF/DOCX/XLSX.
- Large output.
- Binary/NUL handling.
- Cancellation.
- Timeout.

Security

- Secret scanning.
- Dependency audit.
- SAST.
- RLS negative tests.
- Rate limits.
- CSRF/OAuth.
- Log redaction.
- TLS configuration.
- Privilege boundary.

53. CI/CD recommendations

Recommended jobs:

```
lint
typecheck
unit tests
remote-device tests
integration tests
dependency audit
secret scan
license scan
build
package smoke
documentation validation
PDF validation
```

Do not automatically publish modified code under the upstream npm package identity. Use a SIITHSYSTEMS package name if you later distribute a changed implementation.

54. Keeping the fork synchronized

Add upstream:

```
git remote add upstream https://github.com/wonderwhy-er/DesktopCommanderMCP.git
```

Fetch:

```
git fetch upstream --prune --tags
```

Review:

```
git log --oneline --left-right --graph main...upstream/main
```

Recommended process:

```
upstream fetch
-> integration branch
-> automated tests
-> compatibility review
-> docs contract review
-> merge to main
-> synchronize GitLab
```

Do not overwrite SIITHSYSTEMS changes silently.

55. GitHub/GitLab policy

GitHub is useful for:

- Genuine fork relationship.
- Upstream synchronization.
- Public provenance.
- Pull requests.

GitLab can serve as:

- Private SIITHSYSTEMS engineering copy.
- Internal CI/CD.
- Deployment automation.
- Internal project management.

A GitLab copy is not a cross-platform GitHub fork relationship; it is an independent repository with copied Git history.

56. License boundary

DesktopCommanderMCP is MIT licensed.

The official hosted Remote Desktop Commander service is proprietary and its implementation is not included in the public remote-service repository.

Accordingly:

- Build on the MIT local code under its license.
- Do not assume proprietary hosted code is licensed.
- Implement the SIITHSYSTEMS remote server independently using public protocol behavior.
- Preserve required license notices.
- Review trademark/branding before distributing a renamed product.

57. Recommended SIITHSYSTEMS implementation phases

Phase A - Preserve fork

- Keep the fork buildable.
- Add SIITHSYSTEMS docs.
- Track upstream.
- Detect upstream drift.

Phase B - Compatibility harness

Mock:

```
/api/mcp-info  
/device/start  
/device/poll  
Supabase Auth  
mcp_devices  
mcp_remote_calls  
Realtime Broadcast  
Realtime Presence
```

Point the device at the harness with MCP_SERVER_URL.

Phase C - Identity and pairing

Implement accounts, OAuth, device codes, PKCE, refresh-token rotation, and revocation.

Phase D - Device registry/realtime

Implement Presence, heartbeat, RLS, capabilities, device health.

Phase E - Remote MCP gateway

Implement Streamable HTTP MCP, OAuth, tool listing, device routing, call lifecycle, timeout and cancellation.

Phase F - ChatGPT qualification

Validate the complete authorization/redirect/return path plus real filesystem and terminal calls.

Phase G - Hardening

Vault, TLS, rate limits, observability, backup/restore, incident response, deployment rollback, security testing.

58. Proposed SIITHSYSTEMS environment contract

This is a clean-room proposal, not the proprietary service configuration.

```
PUBLIC_BASE_URL=https://mcp.siithsystems.com
MCP_PATH=/mcp
DEVICE_VERIFY_PATH=/verify-device
```

```
SUPABASE_URL=https://example.supabase.co
SUPABASE_PUBLISHABLE_KEY=<PUBLIC>
```

```
OAUTH_ISSUER=https://mcp.siithsystems.com
OAUTH_SIGNING_PRIVATE_KEY=__VAULT__
OAUTH_SIGNING_KEY_ID=primary-2026-01
SESSION_ENCRYPTION_KEY=__VAULT__
DEVICE_CODE_HMAC_KEY=__VAULT__
REFRESH_TOKEN_PEPPER=__VAULT__
```

```
DATABASE_URL=__VAULT__
SUPABASE_SERVICE_ROLE_KEY=__VAULT__
```

```
GOOGLE_OAUTH_CLIENT_ID=<PUBLIC_ID>
GOOGLE_OAUTH_CLIENT_SECRET=__VAULT__
```

```
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USERNAME=<ACCOUNT>
SMTP_PASSWORD=__VAULT__
```

```
LOG_LEVEL=info
TOOL_CALL_TIMEOUT_SECONDS=300
DEVICE_CODE_TTL_SECONDS=900
DEVICE_POLL_INTERVAL_SECONDS=5
```

59. Definition of green

Do not call a self-hosted SIITHSYSTEMS service production-ready until:

```
[ ] Device pairing works from Linux/RHEL
[ ] Headless pairing works
[ ] Pairing codes expire
[ ] PKCE is enforced
[ ] Refresh rotation works
[ ] Device reconnect survives restart
[ ] Revoke immediately removes access
[ ] Multiple devices route correctly
[ ] Private realtime channel is authorized
[ ] RLS cross-user tests pass
[ ] Duplicate calls execute once
[ ] Tool timeout is bounded
[ ] ChatGPT OAuth completes
[ ] ChatGPT can execute a file tool
[ ] ChatGPT can execute a terminal tool
[ ] Stopping the device removes reachability
[ ] Secrets live in Vault/secret manager
[ ] No live secrets exist in Git history
[ ] TLS/security headers pass
[ ] Logs redact access/refresh tokens
[ ] Backup/restore passes
[ ] Rollback passes
[ ] Monitoring/alerting passes
```

60. Quick reference

Official hosted device:

```
npx @wonderwhy-er/desktop-commander@latest remote
```

Debug:

```
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

No session persistence:

```
npx @wonderwhy-er/desktop-commander@latest remote --no-persist-session
```

Logout:

```
npx @wonderwhy-er/desktop-commander@latest remote --logout
```

Custom server:

```
MCP_SERVER_URL=https://mcp.siithsystems.com \  
npx @wonderwhy-er/desktop-commander@latest remote --debug
```

Clone fork:

```
git clone https://github.com/siithsystems/DesktopCommanderMCP.git  
cd DesktopCommanderMCP  
npm ci  
npm run build
```

61. Maintainer source map

Important open-source files:

```
src/index.ts
src/npm-scripts/remote.ts
src/remote-device/README.md
src/remote-device/device.ts
src/remote-device/device-authenticator.ts
src/remote-device/remote-channel.ts
src/remote-device/desktop-commander-integration.ts
src/remote-device/scripts/blocking-offline-update.js
package.json
README.md
SECURITY.md
PRIVACY.md
```

Public remote-service reference repository:

```
desktop-commander/remote-desktop-commander/README.md
desktop-commander/remote-desktop-commander/docs/SETUP.md
desktop-commander/remote-desktop-commander/SECURITY.md
desktop-commander/remote-desktop-commander/server.json
desktop-commander/remote-desktop-commander/.mcp.json
desktop-commander/remote-desktop-commander/plugin.json
desktop-commander/remote-desktop-commander/gemini-extension.json
```

62. Secret-scan reminder

Before future pushes:

```
git grep -nE '(ghp_|glpat-|sk-|BEGIN (RSA|OPENSSH|EC) PRIVATE KEY|password[[:space:]]*=)'
```

Review matches manually and run a proper secret scanner in CI.

Never paste a live token into a commit, issue, Markdown file, PDF, screenshot, or build log.

63. Final engineering conclusion

The SIITHSYSTEMS fork already provides the difficult computer-side foundation:

- Local filesystem and shell execution.
- Remote-device CLI.
- Browser/device pairing client.
- PKCE.
- Session persistence.
- Refresh-token rotation handling.
- Device registry operations.
- Presence and Broadcast.
- Durable call state.
- Duplicate-call defenses.
- Reconnection logic.
- Local MCP child supervision.

The missing piece is the independently hosted remote MCP control plane.

The intended SIITHSYSTEMS architecture is:

```
SIITHSYSTEMS DesktopCommanderMCP fork
      +
independent SIITHSYSTEMS Remote MCP control plane
      +
MCP_SERVER_URL points device to SIITHSYSTEMS
      +
ChatGPT connects to SIITHSYSTEMS /mcp through OAuth
```

This can reproduce the same category of workflow while keeping the implementation and infrastructure under SIITHSYSTEMS control.

64. Change-control note

This guide was prepared against upstream/public material available on 21 September 2026.

Whenever upstream is synchronized, re-check:

```
device auth endpoints
/api/mcp-info response fields
token rotation behavior
table names and fields
realtime channel and event names
capability flags
CLI options
ChatGPT connector instructions
security guidance
```

Update this Markdown guide and its PDF companion together whenever those contracts materially change.